CloudParse

Assignment Report

Detail	Information
Student	Ankit Yadav
Roll number	230101069
Project name	CloudParse
Repository	https://github.com/ankityadav010101/CloudParse

Abstract

CloudParse translates short natural-language cloud and productivity commands into a compact JSON representation with an intent and three optional slots: date, time, and place. The project combines a parallel teacher-labeling pipeline, dataset validation, a compact-model fine-tuning design, a schema-controlled API, Kubernetes manifests, and network security exercises. This report explains the concepts behind the design and records the readings produced locally by the available project workflow.

Goals and contribution

The main goal is to turn a flexible human request into a predictable machine-readable action while making each stage auditable. The repository demonstrates how labelled data, model adaptation, interface validation, deployment configuration, and subnet policy fit into one cloud-oriented system. The current verified results include cleaning all 3,000 supplied commands into 2,728 training records, scoring the local teacher on all 100 held-out validation examples, passing six NACL scenarios, calculating four subnet allocations, and compiling the Python source tree.

System path

Command text -> deterministic label shards -> validated and checksummed dataset -> teacher/student model workflow -> FastAPI parser -> object-backed model files -> Kubernetes deployment -> network policy and service evaluation.

The CloudParse repository contains the source code, experiment results, deployment configuration, and this report.

Scope and example use

The parser is intended for short, single-step requests whose meaning can be expressed by the supported intent set. A request such as “check the weather in Delhi tomorrow” should become a weather intent with a place and date, while leaving unrelated slots null. Keeping the output narrow makes it easier for an application to validate the request, request clarification when information is missing, and route a well-formed command to the correct service. The system does not execute cloud operations itself; it produces a structured interpretation that a separate, permission-aware application can review and use.

Design principles and expected behavior

Three principles guide the implementation. First, the output should be constrained enough for deterministic validation. Second, every training label should remain traceable to the original command and teacher response. Third, model behavior and service behavior should be measured separately. These choices make it easier to identify whether a failure comes from ambiguous language, an incorrect label, a malformed generation, or an unavailable deployment component.

The intended result is an intermediate command frame, not an unrestricted action executor. A downstream application can check permissions, confirm high-impact operations, and ask the user to clarify missing details before performing an action. For example, a calendar-removal intent may identify the request type but still need a date or event identifier from another system. This separation keeps language interpretation useful without treating a model prediction as authorization.

1. Theoretical foundation

Task-oriented language understanding

A command parser is a task-oriented natural-language understanding system. Unlike an open-ended conversational model, it maps an utterance to a constrained representation that a downstream program can validate and act upon. This smaller output space makes it possible to evaluate the system by comparing both its meaning and its structure against labelled examples.

A common pipeline separates the utterance into an intent and a set of slots. The intent identifies the requested operation, such as `alarm_set`, `calendar_query`, or `weather_query`. Slots carry the values needed to perform that operation. For “set an alarm tomorrow at seven,” the intent is `alarm_set`, the date slot is “tomorrow,” the time slot is “seven,” and the place slot is empty. Keeping these fields separate makes the output useful to an API or workflow engine.

Structured prediction and contracts

CloudParse represents a prediction as one object with the exact keys `intent`, `date`, `time`, and `place`. The slot fields can be strings or null. A strict schema is an interface contract: it lets the service reject missing keys, unexpected fields, and invalid value types before a response reaches another component. This is important because syntactically valid JSON can still be semantically incomplete or unsafe to consume.

The system therefore has two distinct quality dimensions. Structural validity asks whether the response can be parsed and satisfies the schema. Semantic accuracy asks whether the intent and every slot match the reference annotation. Exact full-frame accuracy is stricter than individual field accuracy: one wrong field makes the complete command incorrect. Both should be reported because a service needs parseable output and correct meaning.

Sequence-level knowledge distillation

Knowledge distillation transfers behavior from a larger or more capable teacher to a smaller student. In sequence-level distillation, the teacher generates a complete target response for each input, and the student learns to reproduce that response token by token. It uses the teacher's output text and can be implemented with a standard autoregressive training objective.

Logit-level distillation instead compares the teacher's and student's probability distributions over possible next tokens. It can preserve information about alternatives the teacher considered, but requires access to teacher logits. The project uses generated text from a local teacher interface, so sequence-level training is the appropriate implemented path. A limitation is that the student can learn mistakes, omissions, or inconsistent slot conventions present in the teacher's generated labels.

Parameter-efficient adaptation

Fine-tuning a pretrained language model updates its behavior for the command format. Low-Rank Adaptation (LoRA) freezes the base model weights and learns small low-rank update matrices. This reduces the number of trainable parameters and the storage needed for an adaptation, making it useful when compute and memory are limited. The project configuration includes LoRA-based student training, although a trained artifact was not produced in the local run.

Decoding and output control

Generation settings influence the balance between variety and repeatability. A fixed seed and low temperature make teacher labeling more consistent across repeated runs, but they do not guarantee identical output if the model implementation or prompt changes. The prompt therefore specifies the exact fields and asks for a compact object. The cleaner remains the final authority: prompt wording encourages compliance, while validation enforces it. This layered design avoids relying on generation instructions alone for a machine-facing contract.

2. Data engineering and teacher labeling

Data split and leakage control

The supplied kit contains 3,000 command texts for training and a separate 100-example validation file. Training labels are created only from the training pool. The validation examples are reserved for scoring and are not read by the labeling or training path. This separation matters: tuning prompts or labels using evaluation examples would make the measured score optimistic and less representative of unseen requests.

Deterministic parallel labeling

`worker/label.py` divides input records into shards by source-line index modulo the worker count. For two workers, one handles even-numbered lines and the other odd-numbered lines. Deterministic assignment makes the work parallel while keeping each source record attributable to one shard. The Ollama Phi-3.5 teacher is called with temperature zero and a fixed seed; raw response text, source ID, and original command are stored together.

Parallel workers improve throughput, but they introduce coverage and duplicate risks. The cleaning stage therefore checks that shard IDs cover the source pool exactly once. Reproducibility depends on recording the teacher model, prompt, decoding settings, source IDs, and raw responses. Retaining the raw response also allows a reviewer to distinguish a model-format error from a later validation error.

Validation and cleaning rules

`worker/clean.py` parses each raw response and accepts only an object containing the four expected keys. The intent must belong to the supported intent set. A non-null slot must be text copied from the original command. Duplicate IDs or repeated command text are excluded, and retained rows are ordered by source ID. The cleaner then writes compact JSON Lines and a SHA-256 checksum. A checksum detects later changes to the generated dataset; it does not by itself prove the labels are correct.

Local cleaning measurement	Result
Input commands	3,000
Invalid JSON	124
Incorrect key sets	2
Invalid intents	22
Invalid slots	79
Duplicate records	45
Retained training examples	2,728

The rejection counts are emitted by the cleaner's ordered validation checks. The output dataset contains 2,728 examples. Its recorded SHA-256 digest is `5ded5944f8e1d9f3ef3bd5d39e302a5f397465895f80f9d248b39a4d33f029fd`.

Data quality implications

The rejected rows show why automated annotation needs a quality gate before training. Invalid JSON cannot be scored as a structured label; an unknown intent cannot be mapped safely to the application contract; and a slot that does not occur in the command may reflect hallucination or a labeling mismatch. Duplicates can over-weight repeated wording during fine-tuning, so they are removed before training. The current checks target format, vocabulary, and source-text grounding. A future audit should also inspect intent balance, null-slot frequency, and a human-reviewed sample of accepted labels to estimate semantic label quality.

3. Model design and measured evaluation

Student training design

The configured student is EleutherAI Pythia-70M. A small pretrained model offers a lower memory footprint and a shorter inference path than a large general-purpose model, which suits a narrow command schema. It also has less capacity, so its field-level and exact-frame quality must be established empirically rather than assumed from its size.

The training examples pair each command with the teacher's JSON response. The configuration uses seed 42, a maximum of 1,500 examples, sequence length 192, batch size 2, one epoch, learning rate 0.00005, and weight decay 0.01. Separate version names support controlled model releases. The inference package is designed to use a merged adapter so that serving loads one self-contained model directory.

Training setting	Configured value
Student base	EleutherAI Pythia-70M
Maximum training rows	1,500
Maximum sequence length	192 tokens
Batch size / epochs	2 / 1
Learning rate	0.00005
Weight decay / seed	0.01 / 42
Adaptation	LoRA, then merge for serving

Held-out teacher readings

The local Phi-3.5 teacher generated predictions for all 100 validation commands. The project scorer compared each prediction with its reference label and reported zero missing predictions.

Metric	Measured result
JSON validity	94%
Intent accuracy	72%
Date accuracy	81%
Time accuracy	78%
Place accuracy	72%
Exact full-frame accuracy	39%
Validation examples / missing	100 / 0

Interpretation

JSON validity measures contract compliance, not correctness of the extracted command. The 94% result means most outputs could be parsed into the required four-key object, while the lower exact-frame score shows that matching every field at once is harder. The difference between the component scores and the full-frame score is expected: errors across several fields combine, and one incorrect field is enough to make a complete frame incorrect.

These figures establish a measured baseline for the teacher on the supplied held-out set. They are not student scores. A student evaluation should use the same validation file, scorer, normalization rules, and sample count so results can be interpreted consistently. Raw predictions are stored in `results/teacher_predictions.jsonl`; aggregate readings are in `results/teacher_validation.json`.

Evaluation procedure

Each of the 100 predictions is matched to its validation ID. The scorer first checks whether the output is a JSON object with exactly the four required keys. It then compares the intent and each slot with the reference, normalizing whitespace and letter case for text slots. A missing prediction counts as incorrect rather than silently shrinking the denominator. Reporting the sample count alongside percentages makes the result auditable and prevents a high score on a small subset from being mistaken for complete coverage.

4. Inference interface and application security

Request-response boundary

The FastAPI service exposes `POST /parse`, `GET /healthz`, and `GET /readyz`. A parse request carries command text; a successful response returns the exact schema fields and model-version and serving-instance headers. Separating liveness from readiness is useful: a process may be alive while it is still loading its tokenizer and model, but it should not receive user traffic until it can serve valid responses.

The service validates model output before returning it. A malformed JSON string, extra key, missing key, or invalid slot type should not silently pass through as a successful command. The project includes a fail-closed path that returns an error when model output violates the contract. This pattern makes failure explicit and prevents downstream code from treating arbitrary generated text as a trusted action.

Model loading and object storage

Model files are retrieved from S3-compatible object storage at startup. This keeps the service image separate from large model artifacts and allows different model versions to be published without rebuilding application code. A production design should pin each deployment to an immutable artifact version and verify artifact integrity before loading it. Object-store credentials belong in a Kubernetes Secret or an external secret manager, not in source code or a container image.

Container boundary

A container packages the API runtime and its dependencies in a repeatable environment. The image build should use pinned dependencies, minimize unnecessary files, run as a non-root user where practical, and expose only the service port. Health endpoints and model-version headers give operators basic diagnostics. The repository contains a Dockerfile and API implementation, but Docker was not installed in the local environment; image size, endpoint latency, memory use, and request-failure measurements are therefore not claimed.

Failure handling

Model-generated output is untrusted input, even when produced by a local model. The API should parse it, enforce the schema, and return an explicit failure when validation fails. Timeouts and object-store errors should also fail visibly, with useful logs that avoid leaking credentials or user commands. These safeguards separate model quality from service correctness: a service can reject a bad prediction reliably even when the underlying model is imperfect.

Request lifecycle and trust boundaries

A typical request begins when a client submits command text to `/parse`. The service checks the request shape, invokes the selected model, extracts the generated object, validates its keys and value types, and returns either a structured response or a controlled error. Version and serving-instance headers help operators connect a response to the running artifact and replica. Health checks should not reveal credentials, internal prompts, or raw user data.

The main trust boundaries are the client-to-API request, the API-to-model output, and the service-to-object-store connection. Input length limits and timeouts can prevent unexpectedly large requests from consuming excessive resources. Output validation prevents generated text from becoming an unchecked instruction. Object-store access should use least-privilege credentials, and logs should record operational context without storing secrets. These controls complement model evaluation; they do not replace it.

Consistent responses and error handling

A stable API should distinguish an invalid request from an invalid model response and from a temporary dependency failure. These cases can use different status codes and safe error bodies, allowing clients to decide whether to correct input, retry, or stop. The project includes health and readiness routes as well as a fault-injection option for invalid output. Version headers support debugging after an update. In a production service, retries should be bounded and idempotent, and logs should record an error category without exposing secrets or unnecessarily retaining user text.

5. Kubernetes deployment and reliability theory

Declarative workload management

Kubernetes uses desired-state manifests to describe how many application replicas should run and how they are exposed. A Deployment manages replacement Pods and rolling updates; a Service provides a stable address for a changing set of Pods; an Ingress supplies an HTTP entry point. The project manifest configures two replicas, resource requests and limits, probes, a Service, an Ingress, and a CPU-based Horizontal Pod Autoscaler.

Replica count is an availability and capacity setting, not proof of fault tolerance. Two Pods on one physical node can still fail together if that node is unavailable. Production resilience requires appropriate node placement, storage durability, and dependency availability, followed by failure exercises that measure recovery rather than relying only on manifest inspection.

Probes and traffic readiness

A startup probe gives a slow-loading model time to initialize. A readiness probe controls whether a Pod receives traffic through its Service. A liveness probe identifies a process that should be restarted. These checks answer different questions and should not be collapsed into one endpoint: restarting a healthy process that is merely loading a large model can make recovery worse.

The parser readiness endpoint is intended to remain unavailable until model initialization is complete. This prevents requests from reaching an instance that cannot yet make predictions. Probe thresholds should be chosen from observed startup and response times, then tested during cold starts and failure injection.

Scaling and rolling updates

The Horizontal Pod Autoscaler targets 70% CPU and is configured to range from one to two replicas. It adjusts the desired replica count based on observed resource metrics, but it does not guarantee a particular response time. CPU requests influence scheduling and autoscaling calculations; limits constrain resource consumption. A model workload may also be limited by memory, generation length, or tokenizer overhead, so CPU alone may not predict saturation.

A rolling update gradually replaces old Pods with a new version. A safe release verifies readiness before shifting traffic and keeps a rollback path to the prior artifact. Useful measurements include deployment completion time, time to ready, failed requests during rollout, recovery after Pod deletion, and whether the intended model version served each request.

Local deployment evidence

`k8s/app.yaml` contains the application Deployment, Service, Ingress, probes, resource settings, and autoscaler configuration. The local environment did not have `kubectl` or `k3d`, so rollout, recovery, and autoscaling behavior were not executed or measured. The manifest describes the intended operations; it is not presented as runtime evidence.

Operational observability

A useful deployment dashboard should connect replica count, ready replicas, request rate, error rate, and latency percentiles. Resource graphs for CPU and memory help explain whether a slow response comes from saturation, model startup, or dependency delays. Logs should include a request identifier, model version, and error category while avoiding command text unless a retention policy permits it. During a rollout, comparing old and new versions on the same signals helps identify regressions early. Because model loading can dominate cold-start time, readiness duration should be tracked separately from steady-state response time.

6. Network architecture and NACL experiment

CIDR and variable-length subnet allocation

CIDR notation writes a network prefix as an address followed by a prefix length. The prefix fixes the network bits; the remaining bits identify addresses inside the block. A `/22` leaves 10 host bits and contains 1,024 addresses, of which 1,022 are traditionally usable for hosts in a subnet. Variable-length subnet masking assigns a different prefix length to each subnet, reducing unused address space compared with giving every subnet the same size.

The project allocates the largest requirement first from `10.20.0.0/16`. Each new subnet begins on a boundary aligned to its block size. Allocating in descending order avoids leaving a large requirement fragmented across smaller address ranges.

Subnet	Required hosts	CIDR	Usable address range	Broadcast	Usable
app	1,000	10.20.0.0/22	10.20.0.1 - 10.20.3.254	10.20.3.255	1,022
public	250	10.20.4.0/24	10.20.4.1 - 10.20.4.254	10.20.4.255	254
data	100	10.20.5.0/25	10.20.5.1 - 10.20.5.126	10.20.5.127	126
management	20	10.20.5.128/27	10.20.5.129 - 10.20.5.158	10.20.5.159	30

Stateless network access control

A network ACL evaluates numbered rules in order and applies the first matching rule. Specific denies should therefore precede broad allows. The project denies inbound traffic from `198.51.100.0/24` before general web allowances, restricts SSH to management, and explicitly permits ephemeral return traffic where needed.

A NACL is stateless: it does not remember an allowed outbound connection and automatically permit its response. The reverse direction must be permitted by a separate rule. This differs from stateful firewalls, which track connection state. Forgetting return ports can make an otherwise valid request fail after the initial packet was allowed.

Experiment reading

The supplied packet simulator was run with its checking option. All six sample cases matched their expected decisions: 6/6, or 100%. The sample covered allowed HTTPS, denied SSH, application egress, an explicitly denied source range, ephemeral outbound traffic, and a denied database port. The subnet calculator also completed and produced the allocations in the table. Result logs are stored in `results/nacl_sample_check.txt` and `results/subnet_check.txt`.

Reading the network test cases

The packet simulator exercises both positive and negative policy decisions. HTTPS from an allowed source reaches the web rule, while the blocked source range is denied before that allow can match. SSH from a non-management address is denied, and the database port remains closed to the sample inbound request. The outbound application packet and ephemeral-port case demonstrate that permitted service traffic and return-path ports can be expressed explicitly. Testing both directions is valuable because a stateless policy can allow the first packet and still break the reply. Six passing cases verify the examples supplied with the project; they do not exhaust every possible address, protocol, or port combination.

Worked prefix calculation

The application subnet needs at least 1,000 usable addresses. Ten host bits provide 1,024 total addresses; after reserving the network and broadcast addresses, 1,022 remain, so a `/22` is the smallest listed block that meets the requirement. A `/23` would provide only 510 usable addresses. The 250-host public subnet fits a `/24` with 254 usable addresses, while the 100-host data subnet fits a `/25` with 126. The 20-host management subnet fits a `/27` with 30. This largest-first calculation leaves the next subnet aligned at the following valid boundary and makes the allocation easy to audit.

7. Review, Limitations and Conclusion

Reproducibility and measurement design

Reproducibility means another run can identify the input data, model, prompt, configuration, and scoring method that produced a result. CloudParse uses a fixed seed in the labeling and training configuration, stores source IDs, validates shard coverage, orders retained records, and records a SHA-256 digest for the cleaned dataset. Python source compilation passed locally. The repository also includes a Makefile, pinned requirements, unit checks, and a GitHub Actions workflow.

A service-level objective (SLO) turns a quality expectation into a measurable target over a defined period. The project proposes a 99% successful-response objective and a five-second p95 latency objective. Success rate is successful responses divided by attempted requests; p95 is the latency below which 95% of requests complete. A load report should specify duration, concurrency, sample count, error codes, and resource limits so readers can interpret the numbers.

Cost model

The hypothetical rate card can estimate replica cost as hourly price per vCPU and memory multiplied by requested resources and replica count. Cost per 1,000 requests can then be estimated by dividing the hourly cost by requests per hour and scaling to 1,000. This estimate becomes useful only after throughput is measured under the same deployment conditions. No cloud billing or load-test result was produced locally, so this report does not claim a measured service cost or latency.

Limitations and next steps

The teacher achieved 94% JSON validity and 39% exact full-frame accuracy on the 100-command validation set. A trained Pythia student artifact was not produced because the local Python environment lacks Transformers and PEFT. Docker, Kubernetes, and Locust experiments were also unavailable because Docker, `kubectl`, `k3d`, and Locust are not installed. SeaweedFS is configured as a local S3-compatible store; production use should employ durable storage and carefully managed credentials. Future work is to install the training and deployment dependencies, train the student, score it on the same held-out set, and run controlled service-load and recovery experiments.

Conclusion

CloudParse presents a complete design for converting short natural-language requests into validated command frames. The implementation joins deterministic data preparation, a teacher/student model workflow, strict API validation, object-backed model delivery, declarative Kubernetes resources, and subnet security rules. Local experiments confirmed 2,728 cleaned training examples from 3,000 commands, a 100-example teacher evaluation with zero missing predictions, six correct NACL decisions, valid subnet allocations, and successful Python source compilation. The measured baseline and explicit evaluation plan provide a sound foundation for completing student training and operational benchmarking.

Evaluation discipline

The reported numbers are tied to the exact local data and commands recorded in the project results folder. The teacher score describes one run on the supplied 100-example validation set, and the NACL score covers the six included simulator examples. Neither result establishes production performance or behavior on every possible command. Keeping the scope of each measurement explicit makes the findings useful: they form a baseline that can be repeated after model training, API deployment, and load testing.

References

• Bastianelli et al., “SLURP: A Spoken Language Understanding Resource Package,” EMNLP 2020: https://aclanthology.org/2020.emnlp-main.588/.

• Kim and Rush, “Sequence-Level Knowledge Distillation,” EMNLP 2016: https://aclanthology.org/D16-1139/.

• Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” 2021: https://arxiv.org/abs/2106.09685.

• EleutherAI, Pythia-70M model card: https://huggingface.co/EleutherAI/pythia-70m.

• RFC 4632, “Classless Inter-domain Routing”: https://www.rfc-editor.org/rfc/rfc4632.

• Kubernetes, “Configure Liveness, Readiness and Startup Probes”: https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/.

• AWS, “Control subnet traffic with network access control lists”: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-network-acls.html.